

SOLID PRINCIPLES



Agile methodlar kullanılırken hatanın daha hızlı bulunması ve kodun daha efektif çalışması için sahiplenilmiş prensiplerdir. SOLID:

- **Değişiklikleri Kolaylaştırır**
- **Kodun Yeniden Kullanılmasını Sağlar**
- **Testi Kolaylaştırır**
- **Uyum İçinde Çalışır**

1. Single Responsibility

Tek Sorumluluk Prensipli, bir sınıfın, modülün veya fonksiyonun değişmek için yalnızca tek bir nedene sahip olması gerektiğini ifade eder. Nesne yönelimli tasarımda (OOD) bu kural, her bileşenin sistem içerisinde **sadece spesifik bir sorumluluk** alanına odaklanmasını şart koşar.

Amacı ve İşlevi: Bileşenlerin karmaşıklığını (complexity) azaltmak ve sistem modülleri arasındaki bağımlılığı (coupling) minimum seviyeye indirmektir. Bir sınıf birden fazla sorumluluk üstlendiğinde, bu sorumluluklardan birinde yapılacak bir değişiklik diğerlerini de etkileme riski taşır. SRP sayesinde kodun okunabilirliği, test edilebilirliği ve sürdürülebilirliği (maintainability) garanti altına alınır.



```
class Animal {
  constructor(name, feedingType, soundMade) {
    this.name = name;
    this.feedingType = feedingType;
    this.soundMade = soundMade;
  }

  nomenclature() {
    console.log(`The name of the animal is ${this.name}`);
  }

  sound() {
    console.log(`${this.name} ${this.soundMade}s`);
  }

  feeding() {
    console.log(`${this.name} is a ${this.feedingType}`);
  }
}

let elephant = new Animal('Elephant', 'herbivore', 'trumpet');
elephant.nomenclature(); // The name of the animal is Elephant
elephant.sound(); // Elephant trumpets
elephant.feeding(); // Elephant is a herbivore
```

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  nomenclature() {
    console.log(`The name of the animal is ${this.name}`);
  }
}

let animal1 = new Animal('Elephant');
animal1.nomenclature(); // The name of the animal is Elephant

// Sound class
class Sound {
  constructor(name, soundMade) {
    this.name = name;
    this.soundMade = soundMade;
  }

  sound() {
    console.log(`${this.name} ${this.soundMade}s`);
  }
}

let animalSound1 = new Sound('Elephant', 'trumpet');
animalSound1.sound(); //Elephant trumpets

// Feeding class
class Feeding {
  constructor(name, feedingType) {
    this.name = name;
    this.feedingType = feedingType;
  }

  feeding() {
    console.log(`${this.name} is a/an ${this.feedingType}`);
  }
}

let animalFeeding1 = new Feeding('Elephant', 'herbivore');
animalFeeding1.feeding(); // Elephant is a/an herbivore
```

Bu iki koddan sağda olan Single Responsibility prensibine uygun olandır. Sağdaki kodda aynı class yapısı içinde hem isim hem beslenme türü hem de beslenme tipi verilmiştir. Bir yazılımcı bu class yapısında bir problemle karşılaştığı takdirde debug etmesi daha zor olacaktır.

2. Open-Closed Principle (OCP)

Açık/Kapalı Prensibi, yazılım varlıklarının (sınıflar, modüller, fonksiyonlar vb.) **gelişime** (uzantılara) **açık**, ancak **değişime kapalı** olması gerektiğini belirten kuraldır.

Amacı ve İşlevi: Sisteme yeni işlevler veya gereksinimler eklendiğinde, halihazırda test edilmiş ve çalışan mevcut kaynak kodun değiştirilmesini önlemektir. Mevcut kodu değiştirmek, sistemde beklenmedik hatalara (regression) yol açabilir. Bu prensip, genellikle arayüzler (interfaces) ve soyut sınıflar (abstract classes) kullanılarak uygulanan polimorfizm sayesinde gerçekleştirilir. Sisteme yeni bir davranış ekleneceği zaman, mevcut yapıya dokunulmadan yeni alt sınıflar türetilerek entegrasyon sağlanır.

```
// Base class for payment processing
abstract class PaymentProcessor {
    public abstract void processPayment(double amount); // Pure virtual function
}

// Credit card payment processor
class CreditCardPaymentProcessor extends PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment of $" + amount);
    }
}

// PayPal payment processor
class PayPalPaymentProcessor extends PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing PayPal payment of $" + amount);
    }
}

// Function to process payment
public static void processPayment(PaymentProcessor processor, double amount) {
    processor.processPayment(amount);
}

public class Main {
    public static void main(String[] args) {
        CreditCardPaymentProcessor creditCardProcessor = new CreditCardPaymentProcessor();
        PayPalPaymentProcessor paypalProcessor = new PayPalPaymentProcessor();

        processPayment(creditCardProcessor, 100.00); // Processing credit card payment
        processPayment(paypalProcessor, 150.00);     // Processing PayPal payment
    }

    // Function to process payment
    public static void processPayment(PaymentProcessor processor, double amount) {
        processor.processPayment(amount);
    }
}
```

Bu şekilde class override edilerek bir üst classında herhangi bir değişiklik olmadan üzerine ekleme yapılmıştır.

3. Liskov Substitution Principle

Liskov Substitution prensibi, bir yazılım mimarisindeki temel (üst) sınıf nesnelerinin, programın doğruluğunu veya sistemin beklenen işleyişini bozmadan, kendi **alt sınıf nesneleriyle doğrudan değiştirilebilir** olması gerektiğini ifade eder.

Amacı ve İşlevi: Kalıtım (inheritance) hiyerarşisinin anlamsal ve yapısal olarak doğru kurulmasını sağlamaktır. Bir alt sınıf, miras aldığı üst sınıfın sözleşmelerine (contracts) ve beklenen davranışlarına *tamamen* sadık kalmalıdır. Alt sınıflar, üst sınıfların sunduğu metotları ezerken (override ederken) mevcut işleyişi bozan beklenmeyen kısıtlamalar getirmemelidir veya hata (exception) fırlatmamalıdır. Bu prensip ihlal edildiğinde, polimorfik olarak çalışan kod blokları öngörülemeyen hatalar üretir ve yazılımın bütünlüğü bozulur.

```
class Bird extends Animal {  
  fly() {  
    console.log(`${this.name} kanat çırpıyor`);  
  }  
}  
  
const parrot = new Bird('Titi the Parrot');  
makeAnimalSound(parrot); // Titi the Parrot bir ses çıkarıyor  
parrot.fly(); // Titi the Parrot kanat çırpıyor
```

4. Interface Segregation Principle

Arayüz Ayrımı Prensibi, istemcilerin (client sınıflarının) kullanmadıkları ve ihtiyaç duymadıkları metotlara bağımlı olmaya zorlanmamaları gerektiğini ifade eden yazılım tasarım kuralıdır. "Şişman" (fat) ve çok işlevli tek bir arayüz yerine, her biri belirli bir istemci grubuna veya amaca hizmet eden daha küçük, özelleştirilmiş ve yüksek kohezyona sahip arayüzler tasarlanmalıdır.

Amacı ve İşlevi: Sınıfları, yapamadıkları eylemler için metot gövdelerini boş bırakmaya veya `UnsupportedOperationException` gibi hatalar fırlatmaya zorlayan kötü tasarımların önüne geçmektir (ki bu durum aynı zamanda Liskov Prensibi'ni ve Single Responsibility Principle de ihlal eder). Arayüzlerin ayrıştırılması, sistemdeki bağımlılıkları (coupling) azaltır. Bir

arayüzde meydana gelen yapısal bir değişiklik, **yalnızca o arayüzü gerçekten kullanan (implemente eden) sınıfları etkiler**, böylece kodun bakımı ve yeniden kullanılabilirliği kolaylaşır.

```
interface Printer {
    void print();
}

interface Scanner {
    void scan();
}

interface Fax {
    void fax();
}

class BasicPrinter implements Printer {

    public void print() {
        System.out.println("Yazdiriliyor");
    }
}

class AdvancedMachine implements Printer, Scanner, Fax
{
    public void print() {
        System.out.println("Yazdiriliyor");
    }

    public void scan() {
        System.out.println("Taraniyor");
    }

    public void fax() {
        System.out.println("Faks gonderiliyor");
    }
}
```

Kodda görülebileceği üzere, Printer, Fax ve Scanner'ın tümü yazdırma yaptığı için hepsi bu class'ı implemente eder. Fakat BasicPrinter class'ı Printer'a ait olduğu için onu sadece o implemente etmiştir ve gereksiz implementasyondan kaçınılmıştır.

5. Dependency Inversion Principle

Dependency Inversion prensibi, yüksek seviyeli modüllerin (sistemin ana iş mantığını yürüten sınıflar) düşük seviyeli modüllere (donanım, veritabanı, dosya sistemleri gibi detaylar) doğrudan bağımlı olmaması gerektiğini savunur. Bu bağımlılık, her iki tarafın da ortak bir soyutlama (abstraction) üzerinde birleşmesi ile tersine çevrilmelidir.

Amacı ve İşlevi: Sistemin modülleri arasındaki **sıkı bağımlılığı (tight coupling)** ortadan kaldırmaktır. Kodun içerisine doğrudan somut sınıf isimleri (new MySQLDatabase()) yazmak yerine, bu sınıfın arayüzüne (Database) bağımlı olmak, kodun gelecekte başka bir teknolojiye (örneğin PostgreSQL veya MongoDB) geçişini, ana iş mantığını değiştirmeden sağlar. Bu

prensip, sistemin değiştirilebilirliğini ve test edilebilirliğini (mocking/unit testing) üst düzeye taşır.

```
// DIP İHLALİ
class EmailSender {
    public void sendEmail() { /* ... */ }
}

class Notification {
    private EmailSender email = new EmailSender(); // Sıkı
    Bağımlılık!
}
```

Yanlış (Sıkı Bağımlı): Notification sınıfı doğrudan EmailSender sınıfına bağımlı. Yarı SMS göndermek istesen Notification sınıfını değiştirmek zorundasın!

```
// DIP UYGUN
interface MessageSender {
    void send();
}

class EmailSender implements MessageSender {
    public void send() { /* ... */ }
}

class SmsSender implements MessageSender {
    public void send() { /* ... */ }
}

class Notification {
    private MessageSender sender; // Sadece arayüze
    bağımlı!
    public Notification(MessageSender sender) {
        this.sender = sender;
    }
}
```